

Разработка микросервисного сервиса такси

Цель

Разработать backend микросервисного приложения для заказа такси. Система должна демонстрировать разделение ответственности между сервисами, работу с реляционной базой данных, взаимодействие сервисов по сети и реальный сценарий многопоточной обработки.

Сервис моделирует упрощённую платформу заказа такси. Пассажир создаёт поездку, система ищет подходящих водителей, назначает поездку, обновляет статусы и формирует историю. В фоне работает сервис, который обрабатывает очередь заказов и рассылает уведомления.

Микросервисы

1. User Service

Управление пассажирами и водителями.

API

- `POST /passengers` — регистрация пассажира
- `GET /passengers/{id}` — профиль пассажира
- `POST /drivers` — регистрация водителя
- `GET /drivers/{id}` — профиль водителя
- `PATCH /drivers/{id}/status` — обновление статуса водителя

2. Trip Service

Управление поездками: создание, назначение водителя, смена статусов.

API

- `POST /trips` — создать заказ (передаётся `passenger_id`, `origin`, `destination`)
- `GET /trips/{id}` — информация о поездке
- `GET /trips?passenger_id=...` — история поездок пассажира
- `PATCH /trips/{id}/status` — обновить статус (водитель принял, начал, завершил)

Бизнес-логика

- При создании поездки система ищет свободного водителя и назначает его.
- Назначение должно быть атомарным: два одновременных заказа не должны получить одного водителя.
- Смена статуса водителя при принятии и завершении поездки.

3. Notification / Worker Service

Фоновый сервис для асинхронной обработки событий: уведомление водителей и пассажиров об изменениях статуса поездки.

- При старте Notification Service создаётся пул из 3–5 потоков.
- Потоки конкурентно читают задачи из БД и обрабатывают их.
- Необходимо исключить двойную обработку одной задачи двумя потоками одновременно.
- Реализовать корректное завершение воркеров (graceful shutdown) при остановке сервиса.

API

- **POST /notifications** — добавить задачу в очередь (вызывается Trip Service)
- **GET /notifications?trip_id=...** — список уведомлений по поездке

Бизнес-логика

При старте сервиса запускается пул воркеров (например, 4 потока), которые параллельно берут задачи из очереди в БД и обрабатывают их. Каждый воркер:

1. Берёт одну задачу со статусом **PENDING**.
2. Помечает её как "в обработке" (чтобы другой поток не взял ту же задачу).
3. Имитирует отправку уведомления (логирование, задержка или реальный вывод в консоль).
4. Обновляет статус на **SENT** или **FAILED** при ошибке.
5. При неудаче увеличивает счётчик попыток и повторяет позже (максимум 3 попытки).

Примерная схема базы данных

```

passengers      drivers
-----
id (PK)         id (PK)
name            name
email           email
phone           phone
created_at      license_number
                status
                created_at

trips
-----
id (PK)
passenger_id (FK → passengers.id)
driver_id     (FK → drivers.id, nullable)
status
origin
destination
price
created_at
updated_at

notification_tasks
-----
id (PK)
trip_id       (FK → trips.id)
recipient_type -- 'PASSENGER' или 'DRIVER'
```

```
recipient_id
message
status
attempts
created_at
```

- Все таблицы связаны через внешние ключи.
- Индекс на `trips.status` и `notification_tasks.status` для эффективной выборки.
- Транзакция при назначении водителя: SELECT + UPDATE выполняются атомарно, чтобы избежать двойного назначения.

Межсервисное взаимодействие

Откуда	Куда	Событие
Trip Service	User Service	Проверить, существует ли пассажир
Trip Service	User Service	Найти свободного водителя
Trip Service	User Service	Обновить статус водителя
Trip Service	Notification Service	Создать задачу уведомления при смене статуса поездки

Взаимодействие реализуется через очередь сообщений. Каждый сервис работает независимо и не знает о внутренней реализации другого.

Дополнительная бизнес-логика

- Реализовать оценку поездки пассажиром (1–5 звёзд).
- Добавить расчёт цены по формуле: `расстояние * тариф`.
- Подключить Redis для кэширования списка доступных водителей.
- Добавить JWT-аутентификацию.
- Запустить всё через Docker Compose.
- Добавить эндпоинт статистики: количество поездок за день, средняя цена.